

Basic (your first FastAPI route) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `health_route_practice.py` : every real service needs one tiny route that answers "are you up?", and it is the smallest FastAPI app possible — so it's the right place to learn the app, the decorator, and uvicorn before any real logic is involved. **If not:** your first FastAPI route would be the Build Task's `/run-task`, with a database and an agent behind it, and any mistake in the basic setup would be buried under all of that.

Every version below runs from inside `practice/` with `uvicorn health_route_practice:app --reload`, and is checked with `curl localhost:8000/health`. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/health_route_practice.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

Expected output (`curl localhost:8000/health`):

```
{"status": "ok"}
```

This works, is correct, and is genuinely how many real `/health` routes start out. It's missing a typed response model — fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed response with `response_model`

Story: a bare dict can silently change shape (a typo like `"stauts"`) and no one notices until a client breaks. A `response_model` makes FastAPI check every reply and document it. **If not:** the Build Task's replies would have no written contract, and a wrong field name would only be found by the client.

```
# practice/health_route_practice.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    # why: the reply's shape is written down once, in one place
    status: str

# how: FastAPI checks the return value against HealthResponse
@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    """Liveness check: is the process up and able to answer at all."""
    return HealthResponse(status="ok")
```

Expected output:

```
{"status": "ok"}
```

The JSON reply looks identical to Basic's, but `http://localhost:8000/docs` now shows a documented `HealthResponse` schema for this route, and FastAPI would reject (loudly, with a server-side error) any code path that tried to return something that doesn't match `HealthResponse`.

Approach 2 — checking the route with `TestClient`, instead of `curl`

Story: `curl` needs a running server and a second terminal, every time. `TestClient` calls the route inside the same Python process, so one `python` command checks it. **If not:** every later check

in this document (the 422 check, the Build Task's `test_api.py`) would need a live server running first.

```
# practice/health_route_practice.py
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(status="ok")

# when: only when run with "python health_route_practice.py" -
# uvicorn imports this file, so this block does not run there
if __name__ == "__main__":
    # how: no server, no port – the request goes straight to app
    client = TestClient(app)
    response = client.get("/health")
    print(response.status_code)    # expected: 200
    print(response.json())        # expected: {'status': 'ok'}
```

Expected output (`python health_route_practice.py`):

```
200
{'status': 'ok'}
```

`TestClient` needs the `httpx` package (it's in this document's setup line). It runs in milliseconds, because nothing goes over the network.

Approach 3 — routes on an `APIRouter` , in their own file

Story: once there are two or three routes, keeping them all in the file that creates the app gets crowded. `APIRouter` holds routes in a separate file, and the app adds them with one line. **If not:** the Build Task's `routes.py` + `main.py` split would be the first time you ever saw `APIRouter` .

```
# practice/health_routes.py
from fastapi import APIRouter
from pydantic import BaseModel
```

```
# why: a router is a small, separate group of routes –
# it becomes part of an app only when the app includes it
router = APIRouter()

class HealthResponse(BaseModel):
    status: str

# how: same decorator as before, but on router instead of app
@router.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(status="ok")
```

```
# practice/health_route_practice.py
from fastapi import FastAPI
from fastapi.testclient import TestClient
from health_routes import router

app = FastAPI()
# how: every route on the router is now part of the app
app.include_router(router)

if __name__ == "__main__":
    client = TestClient(app)
    response = client.get("/health")
    print(response.status_code)    # expected: 200
    print(response.json())        # expected: {'status': 'ok'}
```

Expected output (python health_route_practice.py):

```
200
{'status': 'ok'}
```

Same reply as Approach 2 — only the file layout changed.

Difference from Basic: Approach 1 adds a real, checked response shape (`HealthResponse`) instead of an unchecked dict. Approach 2 doesn't change the route at all; it adds a quick, repeatable check with `TestClient` instead of a manual `curl` . Approach 3 moves the route onto an `APIRouter` in its own file — the exact `routes.py` / `main.py` split the Build Task's Suggested files use.

Which one should you actually write? Approach 1 is enough for a single `/health` route — it's what almost every real one looks like on day one. Add Approach 2's `TestClient` check as soon as you want to confirm a route works without starting a server. Move to Approach 3's `APIRouter`

layout the moment you're about to add a second or third route — which happens in the very next exercise, and in the Build Task.